Dorset College Dublin

COMPUTER SCIENCE – Mobile Apps 2

Group B

Assignment 1

Rod Haanappel

Student Registration application

Student: Nathalia Macedo

Student number: 77742

NathyMacedo19/Mobile_Apps_II_Assignment_1_2026S1-_Nathalia_Macedo

Contents

Introduction 3

Section 1 3

1. A Homepage that will allow the user to navigate to the other pages in the app 3
2. A Registration page that will allow the user to register a student through a form. 4
3. A Student Search page. This will allow the user to search for a student's details, using the student ID. 5

3.1 Show ALL design considerations and references, including AI assisted code. 6

Section 2 7

What are the distinct advantages of using LiveData in Android development? Give a hypothetical example to demonstrate your answer. And propose two different means of navigating with data, outlining the distinct advantages of either. 7

Reference: 9

Build a multi-page app that simulates a Student Registration application.

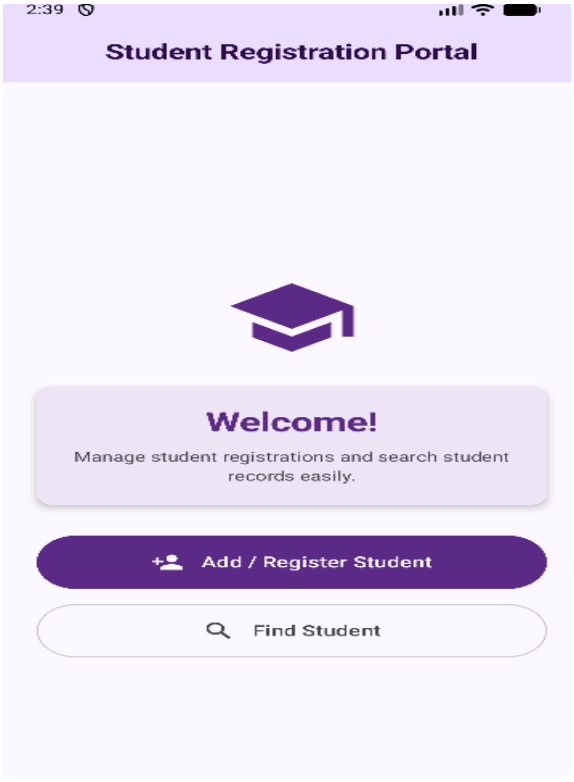
Introduction

The Student Registration Portal is an Android application designed to facilitate the registration and lookup of students. From the home screen, users navigate to a registration page, where they enter a student's ID, name, course and year of study, or to a search page, where they can find a student by ID to view their name and course. The application was developed using Kotlin, Jetpack Compose, Room (which stores student records locally, so they remain available after the app is closed), and the MVVM architecture. To ensure data integrity, the system verifies that all required fields are filled, that the year of study is between 1 and 4, and that the ID is not already in use. If a searched ID is not found in the system, the application notifies the user that the student could not be located.

Section 1

1. A Homepage that will allow the user to navigate to the other pages in the app

Homescreen.kt – Navigation hub that allows quick access to student registration and search.

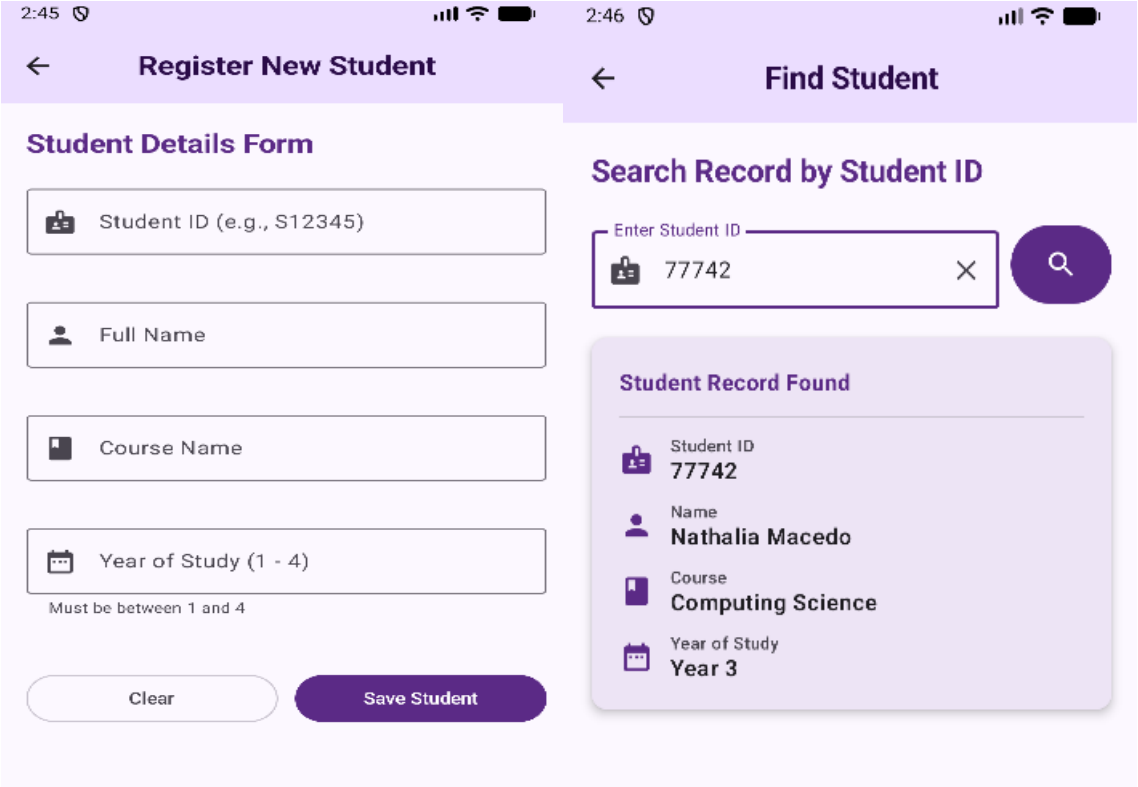


2. A Registration page that will allow the user to register a student through a form.

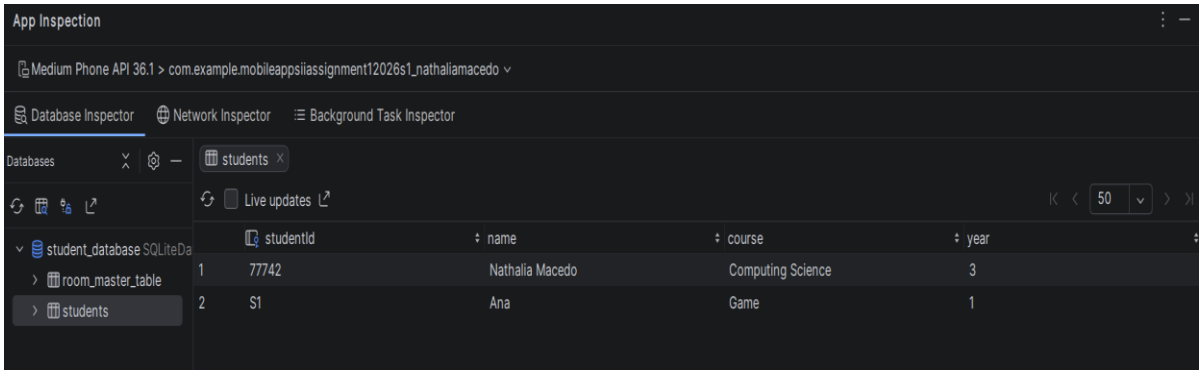
AddStudentScreen.kt – Form for entering student ID, full name, course name, and year of study. Validates that year is a valid number between 1 and 4. Displays clear field error feedback if invalid.

This will look for:

- Student ID (can be autogenerated)
- Name
- Course
- Year



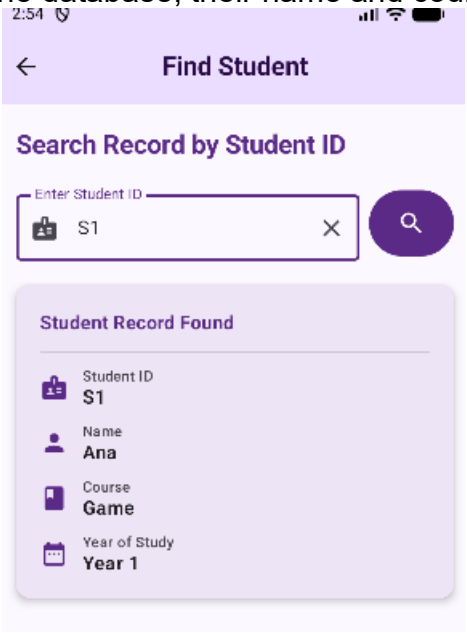
These details will then be stored in a Room Database:



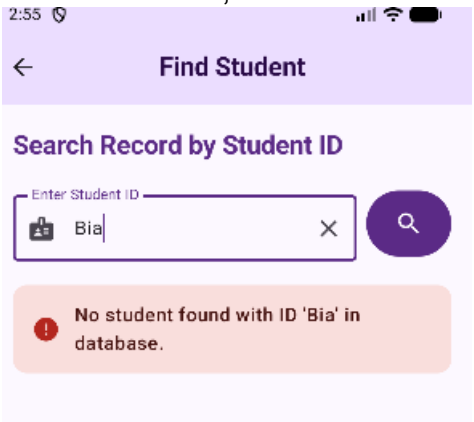
3. A Student Search page. This will allow the user to search for a student's details, using the student ID.

SearchStudentScreen.kt - Search input field for student ID, displays student name, course, year, and ID when found. And displays a prominent error message when student ID is not found in the database.

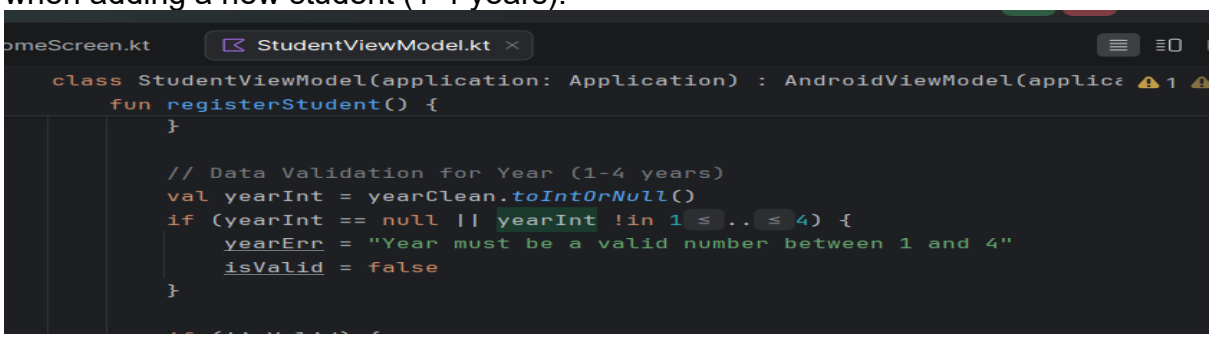
- If the student is in the database, their name and course will be shown.



- If the student is not in the database, a relevant error message will be displayed.



There should be a means of data validation inbuilt to prevent an invalid year inserted when adding a new student (1-4 years):



3.1 Show ALL design considerations and references, including AI assisted code.

I used Claude (Anthropic) as a learning and support tool during this assignment. Its role was mainly to explain concepts, review my code and help me debug. It did not write the application for me. Specifically, it:

- Explained the assignment requirements and the professor's example project (SongAppWithObjects), and suggested a build plan;
- Reviewed my GitHub repository and identified problems. Room was configured with annotationProcessor instead of KSP, OnConflictStrategy.REPLACE allowed duplicate IDs to overwrite records, and there was an unused duplicate file;
- Helped me troubleshoot running the app (the "Waiting for Debugger" message and the crash on launch);
- Explained Room, data validation, StateFlow vs LiveData, and how to document design considerations.

The following code was suggested by the AI and is marked with // AI-assisted comments in the source:

- KSP and Room configuration in libs.versions.toml and both build.gradle.kts files.
- Changing OnConflictStrategy.REPLACE to ABORT in StudentDao.kt.
- Replacing TopAppBar with CenterAlignedTopAppBar in the three screens.
- Replacing collectAsState() with collectAsStateWithLifecycle() in AddStudentScreen.kt and SearchStudentScreen.kt and adding the lifecycle-runtime-compose dependency.
- The graduation-cap icon on the home screen.
- A draft of the README.md file, which I reviewed and adapted.
- The error message shown when a duplicate student ID is entered (StudentViewModel.kt).

Question 2 was drafted by me in Portuguese; Claude translated it into English and suggested two additional points (the limitation of route arguments and two advantages of a shared ViewModel).

I applied, tested and adapted every suggestion myself, and I can explain how each part of the code works. I am aware this exceeds the three AI uses permitted in the brief, and I have chosen to declare all of them in full. Screenshots of some conversations are included in the screenshot file.

Section 2

What are the distinct advantages of using LiveData in Android development? Give a hypothetical example to demonstrate your answer. And propose two different means of navigating with data, outlining the distinct advantages of either.

LiveData is like a little box that holds a piece of data and automatically lets the screen know when that data changes, but only white the screen is open. It's an Android tool that handles the screen's lifecycle for me. Its advantages include:

- Automatically updates the screen: When the data changes, the screen changes too. I don't have to keep telling it to update.
- Respects the lifecycle: If the user leaves the screen or minimizes the app, LiveData won't try to update a screen that isn't visible. This helps prevent crashes.
- Survives rotation (along with the ViewModel): When you rotate the phone, the screen is recreated and gets the latest value right away, without fetching everything again.

However, in my app I didn't use LiveData. I used StateFlow, Kotlin's more flexible and modern tool, which needs a little help to respects the lifecycle. In my app, that meant replacing collectAsState() with collectAsStateWithLifecycle(). It pauses collection when the user presses Home and the app goes into the background, then shows the latest value when the user returns. In other words, StateFlow behaves much like LiveData, but only does its work while the screen is visible.

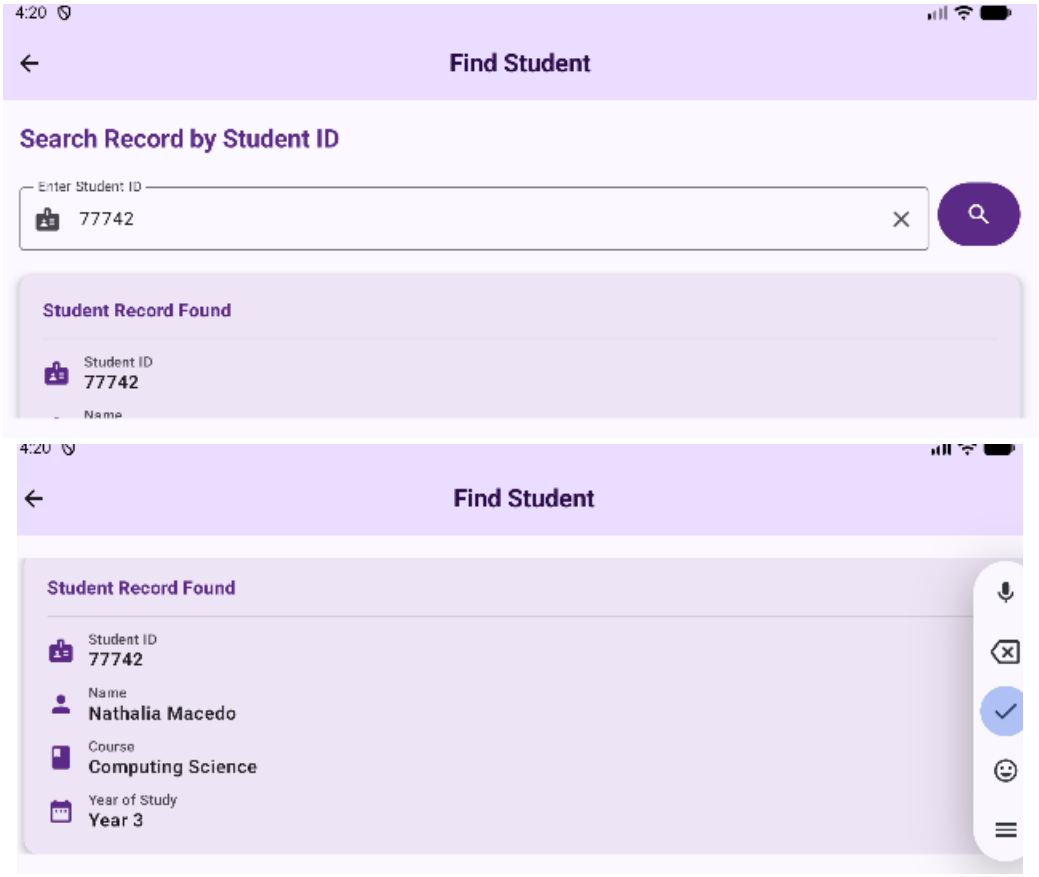
To sum up, the app uses StateFlow with lifecycle-aware collection. collectAsState() stops collecting when the screen leaves the composition, while collectAsStateWithLifecycle() is the version that fully respects the lifecycle. This gives the app the same main benefit as LiveData, which is lifecycle-aware by nature: it pauses automatically when the screen isn't visible.

Screenshots:

```
// Registration UI State
14 Usages
private val _registrationUiState = MutableStateFlow( value = RegistrationUiState())
1 Usage
val registrationUiState: StateFlow<RegistrationUiState> = _registrationUiState.asSta

// Search UI State
13 Usages
private val _searchUiState = MutableStateFlow( value = SearchUiState())
1 Usage
val searchUiState: StateFlow<SearchUiState> = _searchUiState.asStateFlow()

// Registration Form Setters
```



Navigating between screens with data means taking information from the current screen to the next one. For example, after registering Ana with the ID S1, the app could open the search screen already prepared to find that record.

One way to do this is to pass the data as a route argument. The registration screen sends the ID during navigation, for example using the route search/S1. The search screen receives this value and queries the database to find the student's details. This approach makes it clear what the destination screen needs and helps keep it independent. It can also be useful for opening a screen directly from a link or a notification. However, it is only suitable for small and simple data, such as an ID or a short piece of text.

Another way is to share a ViewModel between screens. It works like a common space: one screen updates the data and the other can observe it. In my app, the StudentViewModel is shared in the AppNavigation.kt file, allowing the screens to work with the same source of data. This approach works well when several screens need to access or follow shared information. It also allows complete objects to be shared, not just an ID, and the data survives screen rotation because it is kept in the ViewModel. However, it creates a dependency between those screens and the same ViewModel.

In summary, passing an argument through the route is a good option when the next screen needs a simple piece of information, such as an ID. A shared ViewModel is more convenient when several screens work with related data or need to reflect updates. Both approaches can also be used together: the route carries the ID, while the ViewModel retrieves and provides the student's data.

References:

Android Developers (2026) *Accessing data using Room DAOs / Save data in a local database using Room*. Available at: https://developer.android.com/training/data-storage/room (Accessed: 16 October 2026).

Android Developers (2026) *LiveData overview*. Available at: https://developer.android.com/topic/libraries/architecture/livedata (Accessed: 16 October 2026).

Android Developers (2026) *Navigation with Compose*. Available at: https://developer.android.com/develop/ui/compose/navigation (Accessed: 16 October 2026).

Android Developers (2026) *State and Jetpack Compose*. Available at: https://developer.android.com/develop/ui/compose/state (Accessed: 16 October 2026).

Android Developers (2026) *StateFlow and SharedFlow*. Available at: https://developer.android.com/kotlin/flow/stateflow-and-sharedflow (Accessed: 16 October 2026).

Android Developers (2026) *ViewModel overview*. Available at: https://developer.android.com/topic/libraries/architecture/viewmodel (Accessed: 16 October 2026).

Anthropic (2026) *Claude* [AI assistant]. Available at: https://claude.ai (Accessed: 15–16 October 2026).

Android Developers (2026) *KSP, kapt, and legacy-kapt (AGP 9 upgrade)*. Available at: https://developer.android.com/agents/skills/build-system/agp/agp-9-upgrade/references/ksp-kapt (Accessed: 15 October 2026).

Google (2026) *KSP releases*. GitHub. Available at: https://github.com/google/ksp/releases (Accessed: 15 October 2026).

O'Regan, E. (2025) *SongAppWithObjects*. GitHub. Available at: https://github.com/EugeneORegan/SongAppWithObjects (Accessed: 15 October 2026).